Chapter 16.1: Files and chess

About this page:

Key questions: How to create structure for text files?

Topics: Defining formats for text files

What will I do? Read about text formats and solve the programming tasks

Indicative difficulty level: Moderate

Rough estimate of workload? 5 hours

Point value: S160

Related projects: Chess (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/).

Task: Reading a chunked file (150p + 10p)

The tasks in this and the next section include reading data, which is stored in different formats. In both sections, readable data describes chess game positions. Classes describing the chess game and the outline of a class for interpreting a file are given.

Goals

- Practice reading from a data stream.
- Learn how to store information in a file.
- A major part of the bigger program is written in several phases. In almost every project topic, the program should be able to read and write files. In many project topics, the data format may be simpler.

Intro: File Formats

One key feature of nearly all programs is to have an ability to save information about the program's activities into files, as well as import it into the program. Saved information almost always has a clear structure. The way information is represented in a file is called its *file format*.

A number of different goals influence designing a format or selecting an existing format for a task:

- Readability
- Editability (by hand or by utilities)

- Writing the initial content manually
- Possibility to insert data in between existing data
- How sensitive the format is for errors
- Tasks of the format, as well as the parser (file interpreter)
- Layout
- Extendability of the format
- Speed
- Known formats (e.g., CSV), which allow that data stored by the program can be processed by other programs.

Block-based format

In this assignment, a program reading a block-based format is written. For example, many picture and sound formats work like this. They are easy to read and write with programs. These formats are easily extendable: One can add information that does not have to be understood by all programs that read them, but all programs can still handle the essential information. For example, additional information provided by cameras can be attached to existing image types without changing the original format of the files.

Small task: Examples of other formats

Take a look at these widely-used formats in the following sources and then identify the formats used in the following examples.

- JSON (http://www.json.org/)
- XML (https://www.w3.org/standards/xml/core.html)
- YAML (http://www.yaml.org/)

Note that samples to be identified might only include a part of the file.


```
<bus>
  <passengers>
    <passenger uid="2541">Mary</passenger>
    <passenger uid="2874">Joe</passenger>
  </passengers>
</bus>
```



```
{
  "type": "bus",
  "passengers": [
    { "uid": 2541, "name": "Mary" },
    { "uid": 2874, "name": "Joe" }
  ]
}
```

- ◉ **json**
- ○ xml
- ○ yaml

✔ Correct!

Question 3 **4 / 4**

```
type: bus
passengers:
  - uid: 2541
    name: Mary
  - uid: 2874
    name: Joe
```

- ○ json
- ○ xml
- ◉ **yaml**

✔ Correct!

Submit

It is recommendable to use ready-made libraries for reading and writing data in known formats. In the next section, the purpose of the task is to practice manipulating a format that has been designed to be fairly easy to be handled by humans.

The format used in the assignment

This format of this task has been developed to record the chess game positions. Here we just save the final position, not moves. An example of a file written in the format is given below, followed by a more formal presentation of the format used by the task.

The presented format has been designed to be easily **readable by a program**, having the option to add new blocks while still retaining the possibility to read information with old programs, and good space usage. The file also contains a version number and an identifier that can be used to prevent situations where one tries to read the wrong type of data.

Example

There are no line breaks in the file! In the picture, the text is only divided into shorter sections to make it easier to present.

Location	File contents	Explanation
0	CHESS013	8 Characters: header
8	05072001	8 Characters: date
16	CMT52Lau	Comment block containing
24	ri's rev	52 bytes
32	enge, no	(plus 5 for block header)
40	t doing	
48	great th	
56	is time	
64	either..	
72	.PLR17B5	Player block containing 17 bytes.
80	MarkoKa4	Black. Name has 5 characters: Marko
88	Ra6b3c3P	King a4, Rook a6, Pawn b3, Pawn c3
96	LR13W5La	Player block containing 13 bytes. White.
104	uriKd3Nf	Name has 5 characters: Lauri. King d3,
112	1END00	Knight f1
bytes from beginning of file		End of file

The text above is:

```
CHESS01305072001CMT52Lauri's revenge, not doing great this time either...PLR17B5MarkoKa4Ra6b3c3PLR13W5LauriKd3Nf1END00
```

File Structure

Look at this section while reading the picture above.

The file consists of a title and a set of blocks. More specifically, its structure is as follows:

```
Header: 8 characters
   text "CHESS": 5 characters
   version number: 3 characters

Date: 8 characters
   DDMMYYYY: 8 characters
   (DD is a day between 01 and 31, MM is a month between 01 and 12, YYYY is a year)

One or more blocks of any type, however:
   * The last one is the END block
   * there are exactly 2 PLR blocks (player) and they have different colors
```

Blocks

Each block is structured as follows:

```
Tag         : 3 characters (e.g. CMT, PLR, END)
data length : 2 characters (a number between 00 - 99)
data        : (data length) characters
```

The length of the data indicates the number of characters in the block excluding the five characters taken by the tag and the data length entry itself. In other words, after reading the number, you can safely read the number of characters indicated.

Descriptions of the blocks

Comment block (tag CMT)

Comment block. The content of the entire block is a string that describes the stored position.

Player Block (tag PLR)

Player Block. In a correctly formatted file, there are always two such blocks in different colors.

```
Player color                     : 1 character, B (for black) or W (for white)
Name Length                      : 1 character between 1-9
Name                             : (name length) characters
Player’s pieces in chess notation : until the end of the block
```

Note: The file can also be used to represent chess problems where there can be any number of pieces of any piece type. So you don’t have to think about the number of chess pieces or whether the position is realistic at all.

Chess notation:

The size of the chessboard is 8×8. The board has 8 rows numbered 1–8, and 8 columns marked with small letters a–h. The corners of the board are thus in a1, a8, h1 and h8.

There are 6 different pieces in chess, which we will refer to using their English abbreviations for the purposes of this assignment:

- King `'K'`
- Queen `'Q'`

- Rook `'R'`
- Bishop `'B'`
- Knight `'N'`
- Pawn `''`.

The notation for the king in position a3 is 'Ka3'. The rook in h8 would be written "Rh8". The pawn does not have a character in the notation, so the pawn in b7 would be marked "b7".

The locations of the pieces are listed sequentially in the file without separator marks. If the first character is between a and h, the character described is a pawn, else it indicates the type of the piece.

End block (tag END)

End block. The size of the End block is always 00, and it marks the end of the file, so after reading the end block, the file can be closed.

Other blocks (unknown tag including any three characters)

Unknown blocks are likely to have added features from the newer versions. Of these, only the size of the block is read and then the block is skipped.

Assignment

Implement the method requested in the `game.ChunkIO` object

- `def loadGame (input: Reader): Game`

  Creates a new Game object and returns it at the end of the method. The method reads the game players and the location of the pieces using auxiliary methods. After reading the end block, it returns the game object with the players and the board set in the file. The comment block and possible unknown blocks in the method should be bypassed. When you hopefully test your program, create and close the data stream in your test class, not in this method.

  If there is something wrong with the file you are reading, the method should cast an exception `CorruptedChessFileException`. The given code already throws such an exception in a situation where there is a problem with the data stream. You should throw it if there is a problem with the structure of the read information. *These include, for example, missing the end block, abnormal number of player blocks, overlapping pieces on the board, etc.*

Given program code

Task Package - Contains a code for this and the next section. Chess (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/).

- `game.Board.scala` **(Not submitted)**

  A very simple class that describes the game board. Allows you to place the pieces on the board and view the locations on the board. Includes an auxiliary method for converting column headings to grid indexes.

- `game.Player.scala` **(Not submitted)**

  A very simple class that describes a player. Allows you to set the name and color of the player and read these attributes.

- `game.Game.scala` **(Not submitted)**

A very simple class that puts together the classes you need in the game. Allows to place and retrieve players and the game board.

- `game.Piece.scala` **(Not submitted)**

  A very simple class that describes a chess piece. Chess piece has an owner (player) and a type (King, Knight, etc ...)

- `game.io.CorruptedChessFileException.scala` **(Not submitted)**

  Exception class to inform about problems in reading a file.

- `game.io.test.BrokenReader.scala` **(Not submitted)**

  Auxiliary class for testing error situations.

- `game.io.ChunkIO.scala` **(To be submitted)**

  An auxiliary class that can be used to load and save (in this exercise saving is not implemented) game position from a file.

  In addition to the methods required for the class, you can also implement auxiliary methods. For example, it may be convenient to read different types of blocks in their own methods.

- `game.io.test.IOTest.scala` **(Not submitted, but this can be very useful)**

  Test file skeleton. You may modify this ScalaTest file in order to test that your implementation is working properly. We will discuss testing in more detail in chapters 16.2.

Descriptions of assignment tests

Here is a description of the test data used in the assignment. Based on this, you can customize the tests for yourself to help in debugging.

Example 1

```
CHESS01305072001CMT52Lauri's revenge, not doing great this time either...PLR17B5MarkoKa4Ra6b3c3PLR13W5LauriKd3Nf1END00
```

Example 2

The names of the players and the contents of the comment block are strange, but there is nothing wrong in the game between PLR and CMTEND.

```
CHESS01328121988CMT14END00PLRENDCMTPLR15B3PLRNh4e3Bb6c3PLR11W6CMTENDKa1END00
```

Unknown Blocks

According to the assignment, the file may contain unknown blocks such as ”BAT101234567890” and ”HUH00”. You just have to skip these without processing them.

```
CHESS01305072001BAT101234567890CMT52Lauri's revenge, not doing great this time either...PLR17B5MarkoKa4Ra6b3c3HUH00PLR13W5LauriKd3Nf1END00
```

Broken file

Most of the tests pass automatically, only the names and locations of the pieces may be wrong or the pieces of another player may be missing.

- Testing a broken file reader (passes already with the given initial code).

- Testing for a sudden ending of the file (No need for special action – this passes as long as you don't catch IO exceptions, but let them be changed to `CorruptedChessFileException` in the main method).
- Testing an invalid piece "Ha1".
- Testing an invalid location 'x1'.
- Testing the complete absence of one PLR block.
- Testing for the absence of END block (passes without making changes when handling a sudden end of file). When reading the next block, an exception is thrown `IOException -> CorruptedChessFileException`.
- Finally, test that a correctly formatted file is accepted.

Typical coding errors

- There is some assumed order for the blocks (black or white first, CMT block at the beginning or end, etc.). Note that only the location of the END block is specified.
- The read characters are used directly as indexes (letters '5', 'a' are not converted to numbers). See auxiliary methods in the `Board` class, `extractChunkSize`.
- Forgetting to read the next `chunkHeader`.
- Forgetting to ignore unknown blocks (reading only the header).
- Forgetting to create a game board or set it in the game.
- Creating a player for the board object, and another one with the same name when creating the pieces.

We develop the program in several phases.

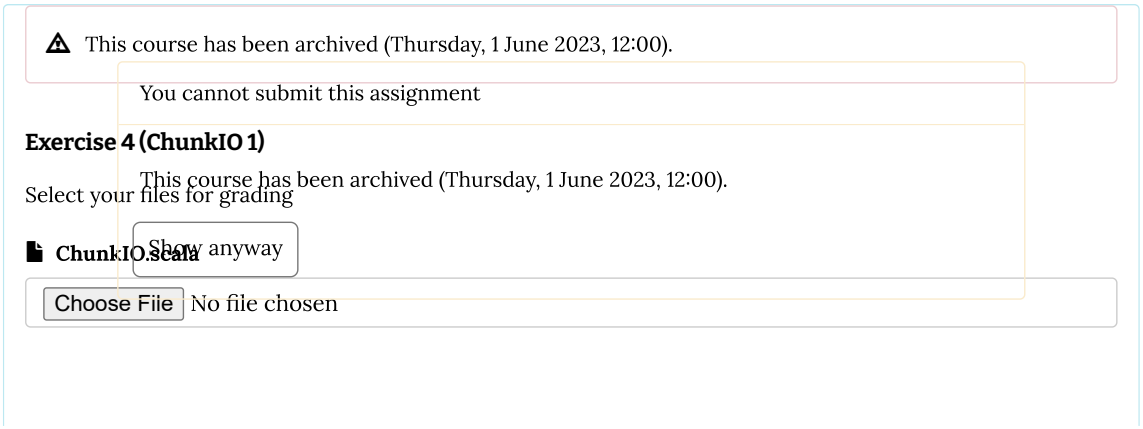
ChunkIO 1 – Reading Block Data

The first exercise is a simple warm-up exercise to help you figure out how to get information out of the file. Your task is to implement in the `loadGame` method the traversal through the different blocks in the file and extracting the data associated with them. More specifically, you need to be able to extract the data blocks of two players (PLR) and the data in the comment block (CMT). There is a `Buffer` for the players and an empty `Option` wrapper for the comment block that you should fill with the data I the file.

`Match-case` structure can be handy for traversing through the different block headers in the file.

Note: It is enough to only extract the data in string format for each block type, e.g. for a player block `PLR17B5MarkoKa4Ra6b3c3` the data portion would be `B5MarkoKa4Ra6b3c3`.

Note: The exercises 1 & 2 handle only files in the correct format and without unknown blocks so for now you do not necessarily need to worry about these cases yet. However you are free to start considering some exception cases already at this point.



Submit

ChunkIO 2 – Building the Game

In the second part, we focus on handling the data from the Player block and using it to build the actual `Game` object. If you have completed part 1, you are now in a nice starting point. In this exercise, you will focus solely on the data you extracted from the player block (PLR), but now you will parse it into the actual game state. You are free to come up with a solution of your own, but here are a few suggestions on how to get started.

The things that you need to extract from the player data are their color, player name and the player's chess pieces and their positions on the board. It might make sense to handle these portions separately and then combine the result. Divide and conquer!

First, you can try to parse together a functioning `Player` object. For that you need the name and the color. After adding players into the game, you can check that they are working correctly by calling the methods `getBlack` and `getWhite` in the `Game` class.

For the chess pieces you can break down the process to determining the piece type, determining its location and placing it on the game board. You can consider doing auxiliary methods for some of these tasks.

`Match-case` is a useful structure all around in this exercise, for example for determining the chess piece type.

Again, we are still dealing with correctly structured files in parts 1 & 2, so for now you can assume that.

Loading assignment...

ChunkIO 3 – Unknown Blocks and Broken Files

Great! Now we have a working chess game in our hands! In the last part, we wrap things up by adding the feature to add new blocks or in other words handling unknown blocks with no features. We will also add detection of broken files by throwing the `CorruptedChessFileException` whenever there is a fault in the file structure.

In each test, the tester checks that the solution also works with correct input: It doesn't help to detect exceptions if the program does not work in a normal setting.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

« Chapter 16.0: Software Testing (https://plus.cs.aalto.fi/studio_...

Chapter 16.2: More about text file formats » (https://plus.cs.aalt...